# Section 5

# ■ Table of Contents

## Section 5 — SageMaker Built-In Algorithms


1. Overview of SageMaker Built-In Algorithms
2. Linear Learner
3. XGBoost
4. LightGBM
5. Seq2Seq
6. DeepAR — Time Series Forecasting
7. BlazingText
8. Object2Vec
9. Object Detection
10. Image Classification
11. Semantic Segmentation
12. Random Cut Forest (Anomaly Detection)
13. Neural Topic Model (NTM)
14. LDA — Latent Dirichlet Allocation
15. KNN — K-Nearest Neighbors
16. K-Means Clustering
17. PCA — Principal Component Analysis
18. Factorization Machines
19. IP Insights
20. Master Comparison Table
21. Instance Type Quick Reference
22. Cheat Sheet + Top 10 Exam Tips

# 1. Overview of SageMaker Built-In Algorithms

## Why Use Built-In Algorithms?

- Pre-optimized, scalable implementations — no custom code required
- Support distributed training across multiple machines & GPUs automatically
- Seamlessly integrate with SageMaker infrastructure (S3, ECR, endpoints)
- Cover supervised, unsupervised, NLP, vision, and time-series tasks
- Many are AWS-optimized versions of popular open-source algorithms

**■ KEY CONCEPT: Each algorithm has four critical dimensions to remember for the exam: Purpose, Input Format, Key Hyperparameters, and Instance Types.**

## Algorithm Categories at a Glance

| Category | Algorithms |
|---|---|
| Supervised — Regression/Classification | Linear Learner, XGBoost, LightGBM, KNN, Factorization Machines |
| Unsupervised — Clustering/Reduction | K-Means, PCA, LDA, Neural Topic Model (NTM) |
| NLP / Text | BlazingText, Seq2Seq, Object2Vec |
| Time Series | DeepAR |
| Computer Vision | Object Detection, Image Classification, Semantic Segmentation |
| Anomaly Detection | Random Cut Forest, IP Insights |

■ EXAM ALERT: The exam loves asking: which algorithm for which task? Know each algorithm's PRIMARY use case cold.

# 2. Linear Learner

## What is it for?

- Linear regression (numeric predictions) and linear classification (binary or multi-class)
- Fits a line (or hyperplane) to training data and makes predictions from it
- Uses a linear threshold function for classification tasks

## Input Format

- RecordIO-wrapped Protobuf — Float32 data ONLY
- CSV — first column must be the label
- Supports both File mode and Pipe mode

■ EXAM ALERT: Float32 only for RecordIO! This is a frequent exam gotcha. Other algorithms accept integer or mixed types.

## How It Works

- Training data MUST be normalized — Linear Learner can auto-normalize
- Input data should be shuffled before training
- Uses Stochastic Gradient Descent (SGD); choose optimizer: Adam, AdaGrad, SGD, etc.
- Trains MULTIPLE models in parallel and selects the most optimal one
- Supports L1 and L2 regularization to prevent overfitting

## Key Hyperparameters

| Hyperparameter | Purpose / Notes |
|---|---|
| balance_multiclass_weights | Gives each class equal importance in the loss function |
| learning_rate | Step size for gradient descent updates |
| mini_batch_size | Batch size for mini-batch SGD |
| L1 | L1 regularization (feature selection / sparsity) |
| wd (weight decay) | L2 regularization to reduce overfitting |
| target_precision | Hold precision at this value, maximize recall — used with recall_at_target_precision |
| target_recall | Hold recall at this value, maximize precision — used with precision_at_target_recall |

## Instance Types

- Single or multi-machine CPU or GPU
- Multi-GPU does NOT help — don't waste money on it

■ EXAM ALERT: Multi-GPU does NOT help Linear Learner. This is a classic exam trap!

# 3. XGBoost — eXtreme Gradient Boosting

## What is it for?

- Classification and Regression using boosted decision trees
- New trees are added to correct errors of previous trees (gradient boosting)
- Uses gradient descent to minimize loss as new trees are added
- Famous for winning Kaggle competitions — fast and accurate

**■ KEY CONCEPT: XGBoost is not built specifically for SageMaker — it's open-source XGBoost adapted for SageMaker. This affects its input format.**

## Input Format

- Primary: CSV or libsvm
- Extended support: RecordIO-Protobuf and Parquet (added by AWS)
- Models serialized/deserialized with Pickle

## How It Works

- Can be used as a framework within notebooks (sagemaker.xgboost)
- Or as a built-in SageMaker algorithm
- Trains an ensemble of decision trees via gradient boosting

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| subsample | Fraction of training data used per tree — prevents overfitting |
| eta (learning rate) | Step size shrinkage to prevent overfitting |
| gamma | Minimum loss reduction to create a partition; higher = more conservative |
| alpha | L1 regularization; higher = more conservative (sparser model) |
| lambda | L2 regularization; higher = more conservative (smoother model) |
| eval_metric | Metric to optimize: AUC, error, rmse — choose based on business need |
| scale_pos_weight | Balance positive/negative weights for imbalanced classes |
| max_depth | Max tree depth — too high causes overfitting |

■ TIP: For imbalanced datasets, set scale_pos_weight = sum(negative cases) / sum(positive cases).

## Instance Types

- Memory-bound algorithm — M5 instances are recommended

- XGBoost 1.2+: Single-instance GPU training available (P2, P3) — must set tree_method=gpu_hist
- XGBoost 1.2-2: Supports P2, P3, G4dn, G5
- XGBoost 1.5+: Distributed GPU training — set use_dask_gpu_training=true and distribution=fully_replicated in TrainingInput
- Distributed GPU only works with CSV or Parquet input

■ EXAM ALERT: XGBoost is MEMORY-bound not compute-bound. Choose M5 over C5 or P-series unless you specifically need GPU.

# 4. LightGBM — Light Gradient Boosting Machine

## What is it for?

- Gradient Boosting Decision Tree — similar to XGBoost and CatBoost
- Extended with GOSS (Gradient-based One-Side Sampling) and EFB (Exclusive Feature Bundling)
- Supports Classification, Regression, and Ranking tasks
- Predicts target variables via an ensemble of simpler models

## Input Format

- Requires TXT or CSV for both training and inference
- Training and optional validation channels may be provided

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| learning_rate | Controls step size for gradient descent |
| num_leaves | Maximum number of leaves per tree |
| feature_fraction | Subset of features used per tree (reduces overfitting) |
| bagging_fraction | Randomly sampled fraction of data per iteration |
| bagging_freq | How often bagging is applied |
| max_depth | Maximum tree depth |
| min_data_in_leaf | Minimum data in one leaf — helps address overfitting |

## Instance Types

- Single or multi-instance CPU training
- Define instances with instance_count hyperparameter
- Memory-bound — choose general purpose over compute-optimized (M5, not C5)
- Ensure sufficient memory for your dataset

■ EXAM ALERT: LightGBM vs XGBoost: Both are gradient boosting tree algorithms. LightGBM adds GOSS + EFB and also supports ranking. Remember: both are memory-bound.

# 5. Seq2Seq — Sequence to Sequence

## What is it for?

- Input is a sequence of tokens → Output is a sequence of tokens
- Key use cases: Machine Translation, Text Summarization, Speech-to-Text
- Implemented with RNNs and CNNs with attention mechanism

## Input Format

- RecordIO-Protobuf ONLY
- Tokens must be INTEGERS (unusual — most algorithms want floats!)
- Start with tokenized text files, convert to protobuf using provided sample code
- Must provide: training data, validation data, AND vocabulary files

■ EXAM ALERT: Seq2Seq uses INTEGER tokens in RecordIO-Protobuf. This is unique — almost all other algorithms use float data.

## How It Works

- Training for machine translation can take DAYS even on SageMaker
- Pre-trained models are available to save time
- Public training datasets exist for specific translation tasks

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| batch_size | Mini-batch size for training |
| optimizer_type | adam, sgd, or rmsprop |
| learning_rate | Step size for optimization |
| num_layers_encoder | Depth of the encoder network |
| num_layers_decoder | Depth of the decoder network |

## Optimization Metrics

- Accuracy — vs. provided validation dataset
- BLEU Score — compares against multiple reference translations
- Perplexity — cross-entropy based metric

## Instance Types

- GPU instances ONLY (e.g., P3)
- Can use only a SINGLE machine for training
- BUT can use multiple GPUs on one machine

■ EXAM ALERT: Seq2Seq: GPU only, single machine only (but multi-GPU on that machine is OK).

# 6. DeepAR — Time Series Forecasting

## What is it for?

- Forecasting one-dimensional time series data
- Uses Recurrent Neural Networks (RNNs)
- Allows training the SAME model over several related time series simultaneously
- Automatically identifies frequencies and seasonality patterns

**■ KEY CONCEPT: DeepAR's key differentiator: it trains ONE model across MANY related time series (e.g., sales of multiple products) unlike traditional methods.**

## Input Format

- JSON Lines format (.jsonl), Gzip or Parquet
- Each record MUST contain: start (timestamp) and target (time series values)
- Optional fields: dynamic_feat (dynamic features), cat (categorical features)

### Example JSON Line:

```
{"start": "2009-11-01 00:00:00", "target": [4.3, 5.1, ...], "cat": [0, 1], "dynamic_feat": [[1.1, 1.2, ...]]}
```

## Best Practices

- Always include the ENTIRE time series for training, testing, and inference
- Use entire dataset as training set; remove last time points for testing
- Do NOT use large values for prediction length (keep under 400)
- Train on MANY time series rather than just one when possible

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| context_length | Number of time points model sees before making a prediction. Can be smaller than seasonalities. |
| epochs | Number of training passes over data |
| mini_batch_size | Batch size for training |
| learning_rate | Step size for gradient descent |
| num_cells | Number of cells in the RNN layers |

## Instance Types

- CPU or GPU, single or multi-machine
- Start with CPU: ml.c4.2xlarge or ml.c4.4xlarge
- Upgrade to GPU only for larger models or mini-batch sizes > 512
- CPU ONLY for inference

■ EXAM ALERT: DeepAR inference is CPU only. Also remember: prediction length should stay below 400 time points.

# 7. BlazingText

## What is it for?

- TWO distinct modes: Text Classification (supervised) and Word2Vec (unsupervised)
- Text Classification: predict labels for a sentence — useful for web search, information retrieval
- Word2Vec: creates vector representations (embeddings) of words; semantically similar words are placed close to each other in vector space
- Word2Vec is NOT an NLP algorithm itself — it creates embeddings FOR NLP algorithms
- Works on individual words only — NOT sentences or documents

**■ KEY CONCEPT: BlazingText = Word2Vec for SageMaker. Used for: machine translation, sentiment analysis. Word2Vec is for individual words, not full text.**

## Input Format

- Supervised (text classification): one sentence per line; first 'word' is __label___
- Augmented manifest text format also supported for supervised mode
- Word2Vec: plain text file with one training sentence per line

### Classification example:

```
__label__4 linux ready for prime time , intel says ...
```

## Word2Vec Modes

- cbow (Continuous Bag of Words) — predicts a word from its context
- skipgram — predicts context words from a target word
- batch_skipgram — distributed computation over many CPU nodes

## Key Hyperparameters

| Mode | Hyperparameters |
|---|---|
| Word2Vec | mode (batch_skipgram/skipgram/cbow), learning_rate, window_size, vector_dim, negative_samples |
| Text Classification | epochs, learning_rate, word_ngrams, vector_dim |

## Instance Types

- cbow/skipgram: recommend single ml.p3.2xlarge (any single CPU or GPU works)
- batch_skipgram: single or multiple CPU instances
- Text classification < 2GB: C5 recommended
- Text classification > 2GB: single GPU (ml.p2.xlarge or ml.p3.2xlarge)

■ EXAM ALERT: Know the two distinct modes of BlazingText and their input formats. Word2Vec works on words — NOT sentences or documents!

# 8. Object2Vec

## What is it for?

- Like Word2Vec but for ARBITRARY objects (not just words)
- Creates low-dimensional dense embeddings of high-dimensional objects
- Use cases: nearest-neighbor search, cluster visualization, genre prediction, recommendations
- Can handle: sentence-sentence, label-sequence, customer-customer, product-product, user-item pairs

## Input Format

- Data must be tokenized into integers
- Training data consists of PAIRS of tokens and/or sequences
- JSON Lines format with label, in0, in1 fields

### Example:

```
{"label": 0, "in0": [6, 17, 606, 19], "in1": [16, 21, 13, 45]}
```

## Architecture

- Two input channels, two encoders, and a comparator
- Encoder choices: Average-pooled embeddings, CNNs, or Bidirectional LSTM
- Comparator followed by a feed-forward neural network

## Key Hyperparameters

- Standard deep learning HPs: dropout, early_stopping, epochs, learning_rate, batch_size, layers, activation function, optimizer, weight_decay
- enc1_network, enc2_network: choose hcnn, bilstm, or pooled_embedding

## Instance Types

- Single machine ONLY (CPU or GPU, multi-GPU OK on one machine)
- ml.m5.2xlarge or ml.p2.xlarge — scale up to ml.m5.4xlarge / ml.m5.12xlarge
- GPU: P2, P3, G4dn, G5
- Inference: ml.p3.2xlarge
- Set INFERENCE_PREFERRED_MODE env variable to optimize for encoder embeddings vs. classification/regression

■ EXAM ALERT: Object2Vec = Word2Vec generalized to any object. Single machine training only (can use multiple GPUs on it).

# 9. Object Detection

## What is it for?

- Identifies ALL objects in an image WITH bounding boxes
- Detects and classifies objects with a single deep neural network
- Classes are accompanied by confidence scores
- Can train from scratch or use pre-trained ImageNet models (transfer learning)

## Two Variants

- MXNet variant: CNN with Single Shot multibox Detector (SSD). Base CNN: VGG-16 or ResNet-50. Supports transfer learning / incremental training.
- TensorFlow variant: Uses ResNet, EfficientNet, MobileNet from TensorFlow Model Garden
- MXNet uses flip, rescale, and jitter internally to avoid overfitting

## Input Format

- MXNet: RecordIO OR image format (JPG/PNG)
- With image format, supply a JSON annotation file for each image (class_id, bounding box coords)

## Key Hyperparameters

- mini_batch_size, learning_rate
- optimizer: sgd, adam, rmsprop, adadelta

## Instance Types

- GPU for training — multi-GPU and multi-machine OK
- ml.p2.xlarge, ml.p2.16xlarge, ml.p3.2xlarge, ml.p3.16xlarge, G4dn, G5
- CPU or GPU for inference: M5, P2, P3, G4dn

■ EXAM ALERT: Object Detection = bounding boxes around each detected object. Different from Image Classification (whole-image label) and Semantic Segmentation (pixel-level).

# 10. Image Classification

## What is it for?

- Assigns one or more labels to an entire image
- Does NOT tell you WHERE objects are — only WHAT objects are in the image
- Simpler than Object Detection (no bounding boxes) and Semantic Segmentation (no pixel masks)

## Two Variants

- MXNet: Full training mode (random weights) OR Transfer learning mode (pre-trained weights, top layer randomized). Default image size: 3-channel 224x224.
- TensorFlow: Uses TF Hub models — MobileNet, Inception, ResNet, EfficientNet. Top classification layer available for fine-tuning.

## Key Hyperparameters

- batch_size, learning_rate, optimizer
- Optimizer-specific: weight_decay, beta1, beta2, eps, gamma
- Slightly different between MXNet and TensorFlow versions

## Instance Types

- GPU for training: ml.p2, p3, g4dn, g5 — multi-GPU and multi-machine OK
- CPU or GPU for inference: M5, P2, P3, G4dn, G5

■ NOTE: Same instance types as Semantic Segmentation. The key difference is the OUTPUT: Image Classification = label; Semantic Segmentation = pixel mask.

# 11. Semantic Segmentation

## What is it for?

- Pixel-level object classification — most granular of the vision algorithms
- Assigns a class to EVERY single pixel in an image
- Produces a segmentation mask as output
- Real-world use: self-driving vehicles, medical imaging diagnostics, robot sensing

## Algorithm & Backbone Choices

- Built on MXNet Gluon and GluonCV
- Algorithms: Fully-Convolutional Network (FCN), Pyramid Scene Parsing (PSP), DeepLabV3
- Backbones: ResNet50 or ResNet101 — both trained on ImageNet
- Supports incremental training and training from scratch

## Input Format

- JPG images and PNG annotations (for both training and validation)
- Label maps to describe annotations
- Augmented manifest image format supported for Pipe mode
- JPG images accepted for inference

## Key Hyperparameters

- epochs, learning_rate, batch_size, optimizer
- algorithm (FCN / PSP / DeepLabV3)
- backbone (ResNet50 / ResNet101)

## Instance Types

- GPU for training: ml.p2, p3, g4dn, g5 — multi-GPU and multi-machine OK
- CPU or GPU for inference: M5, P2, P3, G4dn, G5

■ EXAM ALERT: Vision algorithm comparison: Image Classification = whole-image label | Object Detection = bounding boxes | Semantic Segmentation = pixel-by-pixel mask. Know this triangle!

# 12. Random Cut Forest (RCF) — Anomaly Detection

## What is it for?

- Unsupervised anomaly detection algorithm
- Detects unexpected spikes in time series data, breaks in periodicity, unclassifiable data points
- Assigns an ANOMALY SCORE to each data point
- Higher score = more anomalous
- Developed by Amazon (proprietary algorithm adapted for SageMaker)

**■ KEY CONCEPT: RCF also works in Amazon Kinesis Data Analytics — so it can detect anomalies in STREAMING data in real time, not just batch.**

## How It Works

- Creates a forest of trees where each tree is a partition of the training data
- Measures expected change in complexity of the tree when a new point is added
- High complexity change = the point is anomalous
- Data is sampled randomly before training

## Input Format

- RecordIO-Protobuf or CSV
- File or Pipe mode supported on either
- Optional test channel to compute accuracy, precision, recall, and F1 on labeled data

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| num_trees | Increasing reduces noise in anomaly scores |
| num_samples_per_tree | Should approximate 1/ratio_of_anomalous_to_normal. E.g., if 1% anomalies, use 100. |

## Instance Types

- Does NOT benefit from GPUs — use CPU instances
- Training: M4, C4, or C5
- Inference: ml.c5.xl

■ EXAM ALERT: RCF does NOT use GPUs. Use CPU (M4/C4/C5) for training and ml.c5.xl for inference. Also know: it works with Kinesis for streaming anomaly detection.

# 13. Neural Topic Model (NTM)

## What is it for?

- Organizes documents into TOPICS (unsupervised)
- Classify or summarize documents based on topics
- Example: 'bike', 'car', 'train', 'speed' → topic: transportation
- Uses Neural Variational Inference algorithm

## Input Format

- Four data channels: train (required), validation, test, auxiliary (optional)
- RecordIO-Protobuf or CSV
- Words must be tokenized into INTEGERS
- Every document must have a count for EVERY word in vocabulary (for CSV)
- Auxiliary channel provides the vocabulary
- File or Pipe mode supported

## Key Hyperparameters

- num_topics — you must specify how many topics you want
- Lower mini_batch_size and learning_rate can reduce validation loss (at cost of training time)

## Instance Types

- GPU or CPU — GPU recommended for training
- CPU OK for inference (cheaper)

■ NOTE: NTM vs LDA: Both are topic modeling algorithms. NTM uses deep learning (neural variational inference) and runs on GPU. LDA does not use deep learning and is CPU-based.

# 14. LDA — Latent Dirichlet Allocation

## What is it for?

- Another topic modeling algorithm — similar to NTM but CPU-based
- Unsupervised — topics are unlabeled groupings of documents sharing words
- Not deep learning based
- Can be used for things other than words: e.g., cluster customers based on purchases, harmonic analysis in music

## Input Format

- Train channel (required), optional test channel
- RecordIO-Protobuf or CSV
- Each document must have counts for every word in the vocabulary
- Pipe mode ONLY supported with RecordIO

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| num_topics | Number of topics to discover |
| alpha0 | Initial guess for concentration parameter. Smaller = sparse topic mixtures. Larger (>1.0) = uniform mixtures. |

## Instance Types

- Single-instance CPU training ONLY

■ EXAM ALERT: LDA: CPU only, single instance. NTM: GPU recommended. Both discover topics but with different approaches.

# 15. KNN — K-Nearest Neighbors

## What is it for?

- Simple supervised classification and regression algorithm
- Classification: find K closest points to a sample, return most frequent label
- Regression: find K closest points, return average value

## Input Format

- Train channel contains data; test channel emits accuracy or MSE
- RecordIO-Protobuf or CSV (first column is label)
- File or Pipe mode on either

## How It Works

- Data is first sampled
- Dimensionality reduction stage (sign or fjlt methods) to avoid the curse of dimensionality
- Builds an index for looking up neighbors
- Serializes the model and queries for a given K

## Key Hyperparameters

- K — the number of neighbors (most important!)
- sample_size

## Instance Types

- Training: CPU (ml.m5.2xlarge) or GPU (ml.p2.xlarge)
- Inference: CPU for lower latency; GPU for higher throughput on large batches

# 16. K-Means Clustering

## What is it for?

- Unsupervised clustering — divides data into K groups
- Members of a group are as similar as possible to each other
- Similarity measured by Euclidean distance
- Web-scale K-Means clustering implementation

## Input Format

- Train channel (ShardedByS3Key), optional test (FullyReplicated)
- RecordIO-Protobuf or CSV
- File or Pipe on either

## How It Works

- Every observation mapped to n-dimensional space (n = number of features)
- Optimizes center of K clusters
- Extra cluster centers can be specified to improve accuracy (K = k*x, then reduced back to k)
- Initial centers: random OR k-means++ (tries to make initial clusters far apart)
- Reduce extra clusters using Lloyd's method with k-means++

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| K | Most important! Use elbow method to find optimal K — plot within-cluster sum of squares vs K |
| mini_batch_size | Batch size for training |
| extra_center_factor | Multiplier for extra cluster centers |
| init_method | random or k-means++ |

## Instance Types

- CPU or GPU — CPU recommended
- Only ONE GPU per instance is used even if multiple GPUs available
- If using GPU: ml.g4dn.xlarge
- Supported: p2, p3, g4dn, g4

■ TIP: Use the elbow method to find the best K: plot within-cluster sum of squares vs K and find the 'elbow' point.

# 17. PCA — Principal Component Analysis

## What is it for?

- Dimensionality reduction — reduces high-dimensional data to fewer dimensions
- Projects data while minimizing information loss
- Unsupervised
- First component has the largest possible variability; second has next largest, etc.
- Reduced dimensions called 'components'

## Input Format

- RecordIO-Protobuf or CSV
- File or Pipe mode on either

## How It Works

- Creates a covariance matrix, then applies Singular Value Decomposition (SVD)
- Regular mode: for sparse data, moderate number of observations and features
- Randomized mode: for large observations and features — uses approximation algorithm

## Key Hyperparameters

- algorithm_mode — regular or randomized
- subtract_mean — unbias data before PCA

## Instance Types

- GPU or CPU — depends on specifics of input data

# 18. Factorization Machines

## What is it for?

- Designed for SPARSE data scenarios
- Click prediction, item recommendations
- Users don't interact with most products → sparse user-item matrix
- Supervised: classification or regression
- Limited to PAIR-WISE interactions (e.g., user → item)

## Input Format

- RecordIO-Protobuf with Float32 ONLY
- Sparse data makes CSV impractical

## How It Works

- Finds latent factors to predict classification (click/no-click?) or value (predicted rating?)
- Used in recommender systems context

## Key Hyperparameters

- Initialization methods for bias, factors, and linear terms: uniform, normal, or constant
- Can tune properties of each initialization method

## Instance Types

- CPU or GPU — CPU recommended
- GPU only works with DENSE data — but FM is designed for sparse data, so CPU is the practical choice

■ EXAM ALERT: Factorization Machines: RecordIO-Protobuf Float32 only. GPU only works with dense data, but FM is for sparse data — so effectively CPU only.

# 19. IP Insights

## What is it for?

- Unsupervised learning of IP address usage patterns
- Identifies suspicious behavior from IP addresses
- Use cases: logins from anomalous IPs, accounts creating resources from unusual IP addresses

## Input Format

- CSV only — format: entity, IP
- Usernames and account IDs can be fed directly without pre-processing
- Training channel (required), optional validation (computes AUC score)

## How It Works

- Uses a neural network to learn latent vector representations of entities and IP addresses
- Entities are hashed and embedded — need sufficiently large hash size
- Automatically generates NEGATIVE SAMPLES during training by randomly pairing entities and IPs

## Key Hyperparameters

| Hyperparameter | Purpose |
|---|---|
| num_entity_vectors | Hash size — set to 2x the number of unique entity identifiers |
| vector_dim | Size of embedding vectors. Larger = bigger model. Too large = overfitting. |
| epochs, learning_rate, batch_size | Standard training hyperparameters |

## Instance Types

- CPU or GPU — GPU recommended
- ml.p3.2xlarge or higher; can use multiple GPUs

■ EXAM ALERT: IP Insights = anomaly detection for IP addresses. CSV only input. Automatically generates negative samples. GPU recommended (unlike RCF which is CPU only).

# 20. Master Algorithm Comparison Table

| Algorithm | Type | Primary Use Case | Input Format | GPU? |
|---|---|---|---|---|
| Linear Learner | Supervised | Regression / Classification | RecordIO (Float32), CSV | CPU/GPU (multi-GPU unhelpful) |
| XGBoost | Supervised | Classification / Regression (trees) | CSV, libsvm, RecordIO, Parquet | Optional (M5 preferred) |
| LightGBM | Supervised | Classification / Regression / Ranking | TXT, CSV | No (CPU, M5) |
| Seq2Seq | Supervised | Translation, Summarization, Speech-to-text | RecordIO (INT tokens) | GPU only, single machine |
| DeepAR | Supervised | Time Series Forecasting | JSON Lines (jsonl) | Optional (CPU for inference) |
| BlazingText | Both | Text Classification + Word Embeddings | One sentence/line; plain text | Yes (P3 for cbow/skipgram) |
| Object2Vec | Unsupervised | Embeddings for arbitrary object pairs | JSON Lines (INT pairs) | Yes (single machine) |
| Object Detection | Supervised | Bounding boxes in images | RecordIO / JPG+JSON | GPU required (training) |
| Image Classification | Supervised | Whole-image labeling | RecordIO / image files | GPU required (training) |
| Semantic Segmentation | Supervised | Pixel-level classification | JPG images + PNG annotations | GPU required (training) |
| Random Cut Forest | Unsupervised | Anomaly detection (time series) | RecordIO, CSV | No GPU (CPU only) |
| Neural Topic Model | Unsupervised | Topic modeling (deep learning) | RecordIO, CSV (INT tokens) | GPU recommended |
| LDA | Unsupervised | Topic modeling (non-deep learning) | RecordIO, CSV | No GPU (CPU only, single) |
| KNN | Supervised | Classification / Regression (simple) | RecordIO, CSV | CPU or GPU |
| K-Means | Unsupervised | Clustering | RecordIO, CSV | CPU recommended |
| PCA | Unsupervised | Dimensionality reduction | RecordIO, CSV | CPU or GPU |
| Factorization Machines | Supervised | Sparse data recommendations / clicks | RecordIO Float32 only | CPU (GPU=dense only) |
| IP Insights | Unsupervised | Anomalous IP detection | CSV (entity, IP) | GPU recommended |

# 21. Instance Type Quick Reference

| Algorithm | Training Instances | Inference Instances | Notes |
|---|---|---|---|
| Linear Learner | CPU or GPU (single/multi) | CPU or GPU | Multi-GPU does NOT help |
| XGBoost | M5 (memory-bound); P2/P3/G4dn/G5 for GPU | CPU or GPU | GPU needs tree_method=gpu_hist |
| LightGBM | M5 (memory-bound), multi-instance CPU | CPU | instance_count HP |
| Seq2Seq | P3 (GPU only, single machine) | GPU | Multi-GPU on 1 machine OK |
| DeepAR | ml.c4.2xl/4xl (CPU); GPU for large models | CPU only | GPU only if batch > 512 |
| BlazingText | P3 for cbow/skipgram; multi-CPU for batch_skipgram | CPU or GPU | C5 for small text clf |
| Object2Vec | Single machine: M5 or P2/P3/G4dn/G5 | ml.p3.2xlarge | Multi-GPU on 1 machine OK |
| Object Detection | P2/P3/G4dn/G5 (multi-GPU, multi-machine) | M5, P2, P3, G4dn | GPU required for training |
| Image Classification | P2/P3/G4dn/G5 (multi-GPU, multi-machine) | M5, P2, P3, G4dn, G5 | GPU required for training |
| Semantic Seg. | P2/P3/G4dn/G5 (multi-GPU, multi-machine) | M5, P2, P3, G4dn, G5 | GPU required for training |
| Random Cut Forest | M4, C4, C5 | ml.c5.xl | No GPU at all |
| NTM | GPU recommended | CPU (cheaper) | |
| LDA | Single-instance CPU only | CPU | Cannot scale out |
| KNN | M5 (CPU) or P2 (GPU) | CPU (low latency) / GPU (high throughput) | |
| K-Means | CPU recommended; G4dn if GPU | CPU or GPU | Only 1 GPU used per instance |
| PCA | CPU or GPU | CPU or GPU | Depends on data specifics |
| Factorization Machines | CPU recommended | CPU or GPU | GPU=dense data only |
| IP Insights | ml.p3.2xlarge+ (GPU recommended) | GPU recommended | Multi-GPU OK |

# 22. Cheat Sheet + Top 10 Exam Tips

**Section 5 — SageMaker Built-In Algorithms**

## ■ Quick-Fire Cheat Sheet

| | |
|---|---|
| **Memory-bound algorithms** | XGBoost, LightGBM → use M5 instances |
| **GPU-only algorithm** | Seq2Seq (single machine, can use multi-GPU) |
| **CPU-only algorithm** | LDA (single instance), RCF (no GPU at all) |
| **GPU recommended** | IP Insights, NTM, Object Detection / Classification / Segmentation |
| **Integer token input** | Seq2Seq (RecordIO), NTM (RecordIO/CSV), Object2Vec (JSON Lines) |
| **Float32 RecordIO only** | Linear Learner, Factorization Machines |
| **CSV only** | IP Insights (entity, IP format) |
| **JSON Lines format** | DeepAR (start + target fields required) |
| **Multi-GPU NOT helpful** | Linear Learner |
| **Single machine only** | Seq2Seq, Object2Vec, LDA |
| **Works with Kinesis streaming** | Random Cut Forest |
| **Automatically generates negatives** | IP Insights |
| **Elbow method for K** | K-Means clustering |
| **Pickle serialization** | XGBoost |
| **Word-level only (not sentences)** | BlazingText Word2Vec |
| **Predicts probability distributions** | DeepAR |
| **Two vision variants (MXNet & TF)** | Object Detection, Image Classification |
| **Transfer learning support** | Image Classification, Object Detection |
| **Pixel-level classification** | Semantic Segmentation → segmentation mask |
| **Topic modeling pair** | NTM (deep learning, GPU) vs LDA (non-DL, CPU) |

## ■ Top 10 Exam Tips — Section 5

| | |
|---|---|
| **Tip 1** | Match algorithm to task FIRST. Memorize: bounding boxes = Object Detection, pixel mask = Semantic Segmentation, whole image = Image Classification. |
| **Tip 2** | Know the GPU exceptions: RCF = NO GPU. LDA = single CPU only. Seq2Seq = GPU required. Linear Learner multi-GPU = waste of money. |

| | |
|---|---|
| **Tip 3** | Input format traps: Factorization Machines = Float32 RecordIO ONLY. Seq2Seq = INTEGER tokens. IP Insights = CSV only. DeepAR = JSON Lines. |
| **Tip 4** | For imbalanced data with XGBoost: scale_pos_weight = sum(negative) / sum(positive). For multi-class Linear Learner: balance_multiclass_weights. |
| **Tip 5** | DeepAR: train on MANY time series, not just one. Always include the ENTIRE time series. CPU only for inference. |
| **Tip 6** | BlazingText Word2Vec operates at WORD level only — not sentence or document level. For arbitrary objects, use Object2Vec. |
| **Tip 7** | K-Means: use the elbow method to find K. Only 1 GPU per instance is used. LDA also finds K topics but is CPU-only and non-DL. |
| **Tip 8** | RCF outputs an ANOMALY SCORE per data point and also integrates with Kinesis Analytics for real-time streaming anomaly detection. |
| **Tip 9** | num_samples_per_tree in RCF should ≈ 1 / (ratio of anomalies to normal). If 1% anomalies → num_samples_per_tree ≈ 100. |
| **Tip 10** | XGBoost 1.5+: distributed GPU training is available but requires use_dask_gpu_training=true and distribution=fully_replicated, CSV/Parquet input only. |

---

**Section 5 Complete — Good luck on the AWS MLA-C01 Exam! ■**